

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_4849400f-9d3\agent-a35881c9cda0e4c64.jsonl`
- SHA-256: `3e32278152cc53c92ccb571d6c75615deb9cf819460cffb93ed4d3df284418f9`
- Source modified: `2026-06-21T09:27:31+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_4849400f-9d3`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T09:24:52.597Z)

You are mapping OWNER-GATED decisions for the badminton-highlight-indexer ENGINE serving track (project doc docs/COMPUTE_DECOUPLED_SERVING/README.md). Context: M0-M4 are SHIPPED (the config/input/output/device SEAM layer: deployment.profile+presets, input_backend, output base, DeviceContext). NOW the team is on M5+. The owner wants to BATCH-DECIDE every gate so the maximum independent code path is unblocked. The discipline: ONE isolated PR per milestone off origin/master, default-OFF + golden/cross-OS-gated for any core-caller change, mocked tests for anything not runnable locally (no GPU/cloud here). KEY DISTINCTION you must draw for every gate: separate (a) CODE that CAN be written + merged NOW behind default-OFF with mocked/faked tests (no owner needed) from (b) the precise OWNER PREREQUISITE (real GPU run, GCP spend, counsel/consent review, real prices, OAuth creds, a design pick). For each gate return the structured fields. Ground every claim in real file:line or doc anchors ? read the code, do not guess. Be exhaustive within your area but do not stray outside it.

YOUR AREA: code-reality for M8 cost ledger, M9 edge-auth, M12 gdrive-api OAuth ? how much is pure code now vs owner creds/calibration/counsel.

READ: backend/infrastructure/database.py (the migration ladder + current schema version + any user_id/owner columns on jobs/videos ? v4 user_id was mentioned), docs/archives/past_projects/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md (the cost-attribution design), backend/api/* for any auth/header handling + CORS, backend/storage/gdrive.py (the current MOUNTED-drive backend, to contrast with an OAuth api backend), and grep for any existing Cf-Access / JWT / auth code.

Determine: (M8) can the v_ migration (owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json) + a versioned pricebook DB table (USD internal / INR display, D8) + aggregation + a /api/.../cost endpoint all be written now with placeholder/zero prices the owner later calibrates? (M9) can the Cf-Access JWT signature verification + per-tenant scoping on user_id be written + unit-tested with a fake JWKS now, leaving only the real Cloudflare team-domain/AUD config + the DPA/consent COUNSEL piece to the owner? Split M9 into the codeable-auth half vs the counsel half. (M12) what does the gdrive-api OAuth StorageBackend need from the owner (a Google OAuth client/creds) vs what can be coded against a mocked Drive now? Report M8, M9-auth, M9-consent, M12 as separate gates.

2. assistant (2026-06-21T09:24:55.296Z)

I'll map the M8, M9, and M12 gates by reading the actual code and design docs. Let me start with the key files in parallel.

3. user (2026-06-21T09:24:56.483Z)

```
1      """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3      We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
4      Drive
5      for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
6      requirement ? it
```

```

5      needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
access. So
6      ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
7      your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
code the
8      ``local`` backend uses).
9
10     gdrive://Sharing/clip.mp4 -> <mount>/My Drive/Sharing/clip.mp4
11
12     Why an alias instead of just a raw path? Portability: the mount root differs per machine
13     (Windows ``G:\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
Drive``), so the same
14     ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
15     explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).
16
17     A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
18     ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
19     talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
20     fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21     """
22     from __future__ import annotations
23
24     import glob
25     import os
26     import string
27     from typing import Iterator, Optional
28
29     from backend.storage.base import StorageBackend
30     from backend.storage.local import LocalStorage
31     from backend.storage.ref import StorageRef, StorageStat
32
33     _INSTALL_URL = "https://www.google.com/drive/download/"
34
35     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38     _SYNC_REMEDY = (
39         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42         "downloads the bytes on first access."
43     )
44
45
46     def _not_mounted_msg(tried: Optional[str]) -> str:
47         return (
48             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49             f"  Remedy:\n"
50             f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51             f"  2. Then {_SYNC_REMEDY}\n"
52             f"  3. If your 'My Drive' is not at the default location (Windows G:\My Drive,

```

```

macOS "
53         "~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
54         f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55     )
56
57
58     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59         return (
60             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
61             f"(expected at {os.path.join(root, *rel.split('/')!r}).\n"
62             f"  This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
63             f"  (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
64         )
65
66
67     def _autodetect_mount_root() -> Optional[str]:
68         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
69         Drive for
70         Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
71         Windows
72         ``<letter>:\My Drive`` (`G:` is the install default). Linux/other ? configure
73         explicitly."""
74         home = os.path.expanduser("~")
75         for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage",
76         "GoogleDrive*", "My Drive"))):
77             if os.path.isdir(p):
78                 return p
79         if os.name == "nt":
80             # G is the Drive-for-Desktop default; then scan the rest of the assignable
81             letters.
82             for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
83                 p = f"{letter}:\My Drive"
84                 if os.path.isdir(p):
85                     return p
86         return None
87
88
89     class GDriveStorage(StorageBackend):
90         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
91             root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
92             _autodetect_mount_root()
93             if not root or not os.path.isdir(root):
94                 raise RuntimeError(_not_mounted_msg(root))
95             self._root = os.path.abspath(root)
96             self._local = LocalStorage(base_dir=self._root) # all I/O is plain local-FS
97         under the mount
98
99         def _local_ref(self, ref: StorageRef) -> StorageRef:
100             """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
101             relative to
102             the mount root (``LocalStorage`` joins it under ``base_dir``)."""
103             rel = ref.key.lstrip("/")
104             key = os.path.join(*rel.split("/")) if rel else ""
105             return StorageRef("local", "", key, f"local://{key}")
106
107         def _require_present(self, ref: StorageRef) -> None:
108             """Guard the read paths: a path missing from the mount almost always means the
109             folder
110             isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
111             downstream."""
112             rel = ref.key.lstrip("/")
113             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
114             if not os.path.exists(abs_path):
115                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))

```

```

107     def _to_gdrive_uri(self, abs_path: str) -> str:
108         """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
109         rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
110         return f"gdrive://{rel}"
111
112     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
113         # The file is already local (via the mount): no dst => identity passthrough to
the mount
114         # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
115         self._require_present(ref)
116         return self._local.fetch_to_local(self._local_ref(ref), dst,
overwrite=overwrite)
117
118     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
119         self._local.put(local_path, self._local_ref(ref), content_type=content_type)
120         return ref
121
122     def exists(self, ref) -> bool:
123         return self._local.exists(self._local_ref(ref))
124
125     def stat(self, ref) -> StorageStat:
126         self._require_present(ref)
127         return self._local.stat(self._local_ref(ref))
128
129     def list(self, prefix) -> Iterator[StorageRef]:
130         for r in self._local.list(self._local_ref(prefix)):
131             uri = self._to_gdrive_uri(r.key)
132             yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)
133
134     def delete(self, ref) -> None:
135         self._local.delete(self._local_ref(ref))
136

```

4. user (2026-06-21T09:24:56.737Z)

```

1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):

```

```

30         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31         # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32         conn = sqlite3.connect(self.db_path, timeout=30.0)
33         conn.row_factory = sqlite3.Row
34         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35         conn.execute("PRAGMA foreign_keys = ON")
36         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37         conn.execute("PRAGMA busy_timeout = 30000")
38         try:
39             conn.execute("PRAGMA journal_mode = WAL")
40         except sqlite3.OperationalError:
41             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42         return conn
43
44     def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
45         """Run ONE bool-returning single-statement write inside a connection scope.
46
47         Captures the connect/commit/except-logger.error/return-False boilerplate shared
48         by
49         the bool-returning single-statement writers. ``error_msg`` is the caller's exact
50         per-method message (already formatted with its own ids) ? logged verbatim on a
51         swallowed write fault so each writer keeps its specific log text. Returns True
52         on
53         success, False (after logging ``error_msg``) on any exception."""
54         try:
55             with self._connect() as conn:
56                 conn.cursor().execute(sql, params)
57                 conn.commit()
58             return True
59         except Exception as e:
60             logger.error(f"{error_msg}: {e}")
61             return False
62
63     @staticmethod
64     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
65         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
66
67         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
68         transaction, so an interrupted v3/v4 block can leave the column added while
69         ``user_version`` stays un-bumped ? and the next open would re-run the same
70         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
71         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
72         Swallow
73         ONLY that specific error so the ladder stays crash-safe, matching the re-
74         runnable
75
76         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
77         """
78         try:
79             cursor.execute(ddl)
80         except sqlite3.OperationalError as exc:
81             if "duplicate column name" not in str(exc).lower():
82                 raise
83
84     def _init_db(self):
85         """Initializes tables for videos, play_segments, and events."""
86         with self._connect() as conn:
87             cursor = conn.cursor()
88
89             self._create_base_tables(cursor)
90
91             # Versioned migrations live here. PRAGMA user_version is the schema
92             # contract ? bump it for every change and add an ordered `if version < N`
93             # block below. Tests assert the expected version, so accidental drift fails
94             CI.

```

```

89         version = cursor.execute("PRAGMA user_version").fetchone()[0]
90
91         if version < 1:
92             self._migrate_to_v1(cursor)
93
94         if version < 2:
95             self._migrate_to_v2(cursor)
96
97         if version < 3:
98             self._migrate_to_v3(cursor)
99
100        if version < 4:
101            self._migrate_to_v4(cursor)
102
103        if version < 5:
104            self._migrate_to_v5(cursor)
105        # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
106
107        conn.commit()
108
109    def _create_base_tables(self, cursor):
110        """Create the videos / play_segments / events base tables (idempotent)."""
111        # Videos Table
112        cursor.execute("""
113            CREATE TABLE IF NOT EXISTS videos (
114                id TEXT PRIMARY KEY,
115                filepath TEXT NOT NULL,
116                processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
117                status TEXT NOT NULL,
118                validation_results TEXT
119            )
120        """)
121
122        # Play Segments Table (Extensible metadata JSON)
123        cursor.execute("""
124            CREATE TABLE IF NOT EXISTS play_segments (
125                id INTEGER PRIMARY KEY AUTOINCREMENT,
126                video_id TEXT NOT NULL,
127                start_time REAL NOT NULL,
128                end_time REAL NOT NULL,
129                duration REAL NOT NULL,
130                server TEXT,
131                receiver TEXT,
132                metadata TEXT,
133                FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
134            )
135        """)
136
137        # Events Table
138        cursor.execute("""
139            CREATE TABLE IF NOT EXISTS events (
140                id INTEGER PRIMARY KEY AUTOINCREMENT,
141                segment_id INTEGER NOT NULL,
142                timestamp REAL NOT NULL,
143                type TEXT NOT NULL,
144                player TEXT,
145                details TEXT,
146                FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
147            )
148        """)
149
150    def _migrate_to_v1(self, cursor):
151        # v1: raw candidate detections (pre-confirmation), detector-agnostic.
152        # Common fields are columns; detector-specific metrics go in `stats` JSON,

```

```

153         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
154         cursor.execute("""
155             CREATE TABLE IF NOT EXISTS candidate_segments (
156                 id INTEGER PRIMARY KEY AUTOINCREMENT,
157                 video_id TEXT NOT NULL,
158                 detector TEXT NOT NULL,
159                 chunk_index INTEGER,
160                 start_time REAL NOT NULL,
161                 end_time REAL NOT NULL,
162                 duration REAL NOT NULL,
163                 confidence REAL,
164                 stats TEXT,
165                 detection_params TEXT,
166                 confirmed_segment_id INTEGER,
167                 human_label TEXT,
168                 notes TEXT,
169                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
170                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
171                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
172             )
173         """)
174         cursor.execute("""
175             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
176             ON candidate_segments(video_id, detector)
177         """)
178         cursor.execute("PRAGMA user_version = 1")
179
180     def _migrate_to_v2(self, cursor):
181         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
182         # /api/process request. `status` is the EXECUTION state of the job
183         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
184         # that failed validation) lives inside `result` JSON as result["status"].
185         # `result` is the full sync-era response dict (incl. report paths), so the
186         # observability report is the job's artifact, per the plan.
187         cursor.execute("""
188             CREATE TABLE IF NOT EXISTS jobs (
189                 id TEXT PRIMARY KEY,
190                 video_path TEXT NOT NULL,
191                 video_id TEXT,
192                 segmenter TEXT,
193                 player_pool TEXT,
194                 max_frames INTEGER,
195                 status TEXT NOT NULL DEFAULT 'queued',
196                 progress TEXT,
197                 error TEXT,
198                 result TEXT,
199                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
200                 started_at TIMESTAMP,
201                 finished_at TIMESTAMP
202             )
203         """)
204         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
205         cursor.execute("PRAGMA user_version = 2")
206
207     def _migrate_to_v3(self, cursor):
208         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
209         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
210         # due
211         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
212         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
213         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
214         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
215         # them.
216         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")

```

```

215         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
216         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
217         cursor.execute("PRAGMA user_version = 3")
218
219     def _migrate_to_v4(self, cursor):
220         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
221         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
222         # path can scope rows to an owner. **NULL = local install = byte-identical to
223         # today** ? every existing row stays NULL and every existing query (which never
224         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
225         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
them.
226         # Additive ONLY: no served-behavior change rides on this slice.
227         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
228         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
229         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
230         # of index churn while making the eventual per-user lookups cheap.
231         cursor.execute(
232             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
233             "WHERE user_id IS NOT NULL")
234         cursor.execute(
235             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
236             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
237         cursor.execute("PRAGMA user_version = 4")
238
239     def _migrate_to_v5(self, cursor):
240         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
241         # version): the resolved per-user `fusion_config` overlay + an INLINE
242         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
243         # and provenance that earned it. `is_active` pins the one row the serving bridge
244         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
245         # only persists/loads it ? NOTHING reads it on the served path yet.
246         #
247         #   user_id           ? owner of the model (NULL allowed = the local install)
248         #   version           ? monotonic per-user model version
249         #   baseline_version  ? the shared baseline this was fit against (upgrade
switch)
250         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
251         #   fusion_config     ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
252         #   classifier_json   ? JSON LogisticRegression.to_dict() (None = config-only
model)
253         #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
promoted)
254         #   n_videos_at_fit   ? held-out-user corpus size when fit (min_n trust floor
input)
255         #   is_active         ? the one resolved row for this user (partial-unique
below)
256         cursor.execute("""
257             CREATE TABLE IF NOT EXISTS user_models (
258                 id INTEGER PRIMARY KEY AUTOINCREMENT,
259                 user_id TEXT,
260                 version INTEGER NOT NULL DEFAULT 1,
261                 baseline_version TEXT,
262                 feature_schema_hash TEXT,
263                 fusion_config TEXT,
264                 classifier_json TEXT,
265                 promotion_evidence TEXT,

```



```

266         n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
267         is_active INTEGER NOT NULL DEFAULT 0,
268         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
269     )
270     """
271     # One model version per user (NULLs compare distinct in SQLite, so the local
272     # install can still carry multiple versioned rows; the active-row guard below
273     # is what enforces a single served model).
274     cursor.execute("""
275         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
276         ON user_models(user_id, version)
277     """)
278     # At most ONE active model per user: a partial-unique index over the active
rows.
279     # (SQLite treats NULL user_id rows as distinct, so the local install's single
280     # active row is still guarded ? there is one local user.)
281     cursor.execute("""
282         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
283         ON user_models(user_id) WHERE is_active = 1
284     """)
285     cursor.execute("PRAGMA user_version = 5")
286
287     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
288         """Register or update a video row WITHOUT touching its child segments.
289
290         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
291         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
292         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
293         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
294         row in place; clearing segments is now an explicit, deliberate operation
295         (clear_video_data / prune_segments).
296         """
297         return self._execute_write(
298             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
299             "ON CONFLICT(id) DO UPDATE SET "
300             "filepath=excluded.filepath, status=excluded.status, "
301             "validation_results=excluded.validation_results",
302             (video_id, filepath, status, json.dumps(validation_results)),
303             "Failed to add video",
304         )
305
306     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
307         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
308
309         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
310         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
311         """
312         with self._connect() as conn:
313             cur = conn.cursor()
314             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
315                             (video_id,)).fetchone()[0]
316             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
317                             (video_id,)).fetchone()[0]
318             return int(p), int(c)
319
320     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
play_after: Optional[int] = None, cand_upto: Optional[int] =
321

```

```

None,
322             cand_after: Optional[int] = None) -> None:
323         """Delete play/candidate segments by id range (events cascade from
play_segments).
324
325         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
326         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
327         and keep the OLD ones when the run failed/produced nothing (`*_after`).
328         """
329         with self._connect() as conn:
330             cur = conn.cursor()
331             if play_upto is not None:
332                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
333             if play_after is not None:
334                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
335             if cand_upto is not None:
336                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
337             if cand_after is not None:
338                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
339             conn.commit()
340
341         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
342             try:
343                 with self._connect() as conn:
344                     cursor = conn.cursor()
345                     cursor.execute(
346                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
347                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
348                     )
349                     conn.commit()
350                     return cursor.lastrowid
351             except Exception as e:
352                 logger.error(f"Failed to add play segment: {e}")
353                 return None
354
355         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
356             return self._execute_write(
357                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
358                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
359                 "Failed to add event",
360             )
361
362         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
363                                 chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
364                                 stats: Optional[Dict[str, Any]] = None,
365                                 detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
366             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
367             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
368             try:
369                 with self._connect() as conn:

```

```

370         cursor = conn.cursor()
371         cursor.execute(
372             """INSERT INTO candidate_segments
373                 (video_id, detector, chunk_index, start_time, end_time, duration,
374                 confidence, stats, detection_params)
375                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
376             (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
377             confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
378         )
379         conn.commit()
380         return cursor.lastrowid
381     except Exception as e:
382         logger.error(f"Failed to add candidate segment: {e}")
383         return None
384
385     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
386         """Record that a candidate detection was confirmed as a given play_segment."""
387         return self._execute_write(
388             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
389             (segment_id, candidate_id),
390             f"Failed to link candidate {candidate_id} to segment {segment_id}",
391         )
392
393     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
394                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
395         """Fetch candidate detections for the annotation / fine-tuning workflow.
396         `stats` and `detection_params` are deserialized from JSON."""
397         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
398         params: List[Any] = [video_id]
399
400         if detector:
401             query += " AND detector = ?"
402             params.append(detector)
403         if only_unlabeled:
404             query += " AND human_label IS NULL"
405
406         query += " ORDER BY start_time"
407
408         candidates = []
409         with self._connect() as conn:
410             rows = conn.cursor().execute(query, params).fetchall()
411             for row in rows:
412                 d = dict(row)
413                 d["stats"] = json.loads(d["stats"] or "{}")
414                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
415                 candidates.append(d)
416         return candidates
417
418     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
419         with self._connect() as conn:
420             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
421             if row:
422                 d = dict(row)
423                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
424                 return d
425             return None
426
427     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
428         """
429         Dynamically queries play segments based on filters:
430         - duration_min: float

```

```

431         - server: string
432         - receiver: string
433         - event_type: string (e.g. smash, foul)
434         """
435         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
436         params = [video_id]
437
438         if filters.get("duration_min"):
439             query += " AND r.duration >= ?"
440             params.append(filters["duration_min"])
441
442         if filters.get("server"):
443             query += " AND r.server = ?"
444             params.append(filters["server"])
445
446         if filters.get("receiver"):
447             query += " AND r.receiver = ?"
448             params.append(filters["receiver"])
449
450         if filters.get("event_type"):
451             # Check if there is an event of a specific type inside this segment
452             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
453             params.append(filters["event_type"])
454
455         segments_list = []
456         with self._connect() as conn:
457             cursor = conn.cursor()
458             rows = cursor.execute(query, params).fetchall()
459
460             for row in rows:
461                 r_dict = dict(row)
462                 r_id = r_dict["id"]
463                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
464
465                 # Fetch associated events
466                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
467                 r_dict["events"] = []
468                 for er in event_rows:
469                     e_dict = dict(er)
470                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
471                     r_dict["events"].append(e_dict)
472
473                 segments_list.append(r_dict)
474
475         return segments_list
476
477     def clear_video_data(self, video_id: str):
478         with self._connect() as conn:
479             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
480             conn.commit()
481
482     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
483     # Writers follow the repo convention (swallow + logger.error, return bool); the
484     # worker logs loudly on a failed transition so a job can't silently wedge.
485
486     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487                   player_pool: Optional[List[str]] = None,
488                   max_frames: Optional[int] = None,
489                   policy: Optional[Dict[str, Any]] = None) -> bool:
490         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
491         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
492         return self._execute_write(

```

```

493         "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy) "
494         "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
495         (job_id, video_path, segmenter,
496         json.dumps(player_pool) if player_pool is not None else None,
497         max_frames, json.dumps(policy) if policy is not None else None),
498         f"Failed to create job {job_id}",
499     )
500
501     def mark_job_running(self, job_id: str) -> bool:
502         return self._execute_write(
503             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
504             "progress='starting' WHERE id = ?", (job_id,),
505             f"Failed to mark job {job_id} running",
506         )
507
508     def set_job_progress(self, job_id: str, progress: str) -> bool:
509         return self._execute_write(
510             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
511             f"Failed to set progress for job {job_id}",
512         )
513
514     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
515         return self._execute_write(
516             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
517             f"Failed to set video_id for job {job_id}",
518         )
519
520     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
521         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
522         validation) is inside `result` ? job status 'done' means 'the worker
523         finished'."""
524         return self._execute_write(
525             "UPDATE jobs SET status='done', progress='finished', result=?, "
526             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
527             (json.dumps(result), job_id),
528             f"Failed to mark job {job_id} done",
529         )
530
531     def mark_job_failed(self, job_id: str, error: str) -> bool:
532         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
533         return self._execute_write(
534             "UPDATE jobs SET status='failed', error=?, "
535             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
536             f"Failed to mark job {job_id} failed",
537         )
538
539     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
540         """Fetch one job; `result`/`player_pool` JSON deserialized."""
541         with self._connect() as conn:
542             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
543             (job_id,)).fetchone()
544             if not row:
545                 return None
546             d = dict(row)
547             d["result"] = json.loads(d["result"]) if d["result"] else None
548             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
549             None
550             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
551             return d
552
553     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
554     -----
555     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
556         """Attach/replace a job's JSON ExecutionPolicy."""

```

```

553         return self._execute_write(
554             "UPDATE jobs SET policy=? WHERE id = ?",
555             (json.dumps(policy) if policy is not None else None, job_id),
556             f"Failed to set policy for job {job_id}",
557         )
558
559     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
560         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
561         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
562         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
563         return self._execute_write(
564             "UPDATE jobs SET status='waiting', "
565             "next_poll_at=datetime('now', ?), "
566             "progress=COALESCE(?, progress) WHERE id = ?",
567             (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
568             f"Failed to set job {job_id} waiting",
569         )
570
571     def seconds_until_next_due(self) -> Optional[float]:
572         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
573         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
574         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
575         try:
576             with self._connect() as conn:
577                 row = conn.cursor().execute(
578                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
580                     "FROM jobs WHERE status='waiting'").fetchone()
581                 if row is None or row["secs"] is None:
582                     return None
583                 return max(0.0, float(row["secs"]))
584         except Exception as e:
585             logger.error(f"Failed to compute next-due delay: {e}")
586             return None
587
588     def claim_due_job(self) -> Optional[Dict[str, Any]]:
589         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
590         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
591         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
592         first. Returns the claimed job dict, or None if nothing is runnable."""
593         try:
594             with self._connect() as conn:
595                 cur = conn.cursor()
596                 row = cur.execute(
597                     "SELECT id, status FROM jobs WHERE status='queued' "
598                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
600                 if not row:
601                     return None
602                 cur.execute(
603                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
604                     "WHERE id=? AND status=?", (row["id"], row["status"]))

```

```

605         conn.commit()
606         if cur.rowcount != 1:
607             return None # lost the race to a concurrent
worker
608         return self.get_job(row["id"])
609     except Exception as e:
610         logger.error(f"Failed to claim a due job: {e}")
611         return None
612
613     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
614         """Crash recovery: any job still queued/running from a previous process is dead
615         (the executor is in-process). Mark failed so clients see a terminal state, not a
616         hang. Called once at startup. Returns the number of jobs swept."""
617         try:
618             with self._connect() as conn:
619                 cur = conn.cursor()
620                 cur.execute(
621                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
622                     "WHERE status IN ('queued', 'running')", (reason,))
623                 conn.commit()
624                 return cur.rowcount
625         except Exception as e:
626             logger.error(f"Failed to sweep stale jobs: {e}")
627             return 0
628
629     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
630     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
631     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
632     # serving.py) resolves them, and wiring that bridge into the production decision
seam
633     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
634     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
635     # JSON-serialised dicts; the rest are scalar columns.
636
637     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
638                           baseline_version: Optional[str] = None,
639                           feature_schema_hash: Optional[str] = None,
640                           fusion_config: Optional[Dict[str, Any]] = None,
641                           classifier_json: Optional[Dict[str, Any]] = None,
642                           promotion_evidence: Optional[Dict[str, Any]] = None,
643                           n_videos_at_fit: int = 0,
644                           is_active: bool = False) -> Optional[int]:
645         """Insert or replace ONE per-user model row keyed by `(user_id, version)`.
646
647         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
648         the
649         rowid). When `is_active` is True this row becomes the user's single active
650         model ?
651         any previously-active row for the SAME user is first cleared, so the
652         `idx_user_models_one_active` partial-unique invariant always holds (one active
653         model
654         per user). Returns the row id, or None on failure.
655         """
656         try:
657             with self._connect() as conn:
658                 cur = conn.cursor()
659                 if is_active:
660                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
661                     if user_id is None:
662                         cur.execute(
663                             "UPDATE user_models SET is_active = 0 "
664                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
665                             (version,))

```

```

663         else:
664             cur.execute(
665                 "UPDATE user_models SET is_active = 0 "
666                 "WHERE user_id = ? AND is_active = 1 AND version != ?",
667                 (user_id, version))
668     cur.execute(
669         """INSERT INTO user_models
670         (user_id, version, baseline_version, feature_schema_hash,
671         fusion_config, classifier_json, promotion_evidence,
672         n_videos_at_fit, is_active)
673         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
674         ON CONFLICT(user_id, version) DO UPDATE SET
675         baseline_version=excluded.baseline_version,
676         feature_schema_hash=excluded.feature_schema_hash,
677         fusion_config=excluded.fusion_config,
678         classifier_json=excluded.classifier_json,
679         promotion_evidence=excluded.promotion_evidence,
680         n_videos_at_fit=excluded.n_videos_at_fit,
681         is_active=excluded.is_active""",
682         (user_id, version, baseline_version, feature_schema_hash,
683         json.dumps(fusion_config) if fusion_config is not None else None,
684         json.dumps(classifier_json) if classifier_json is not None else
685         None,
686         json.dumps(promotion_evidence) if promotion_evidence is not None
687         else None,
688         n_videos_at_fit, 1 if is_active else 0))
689     conn.commit()
690     rid = cur.execute(
691         "SELECT id FROM user_models WHERE "
692         + ("user_id IS NULL" if user_id is None else "user_id = ?")
693         + " AND version = ?",
694         ((version,) if user_id is None else (user_id, version))).fetchone()
695     return int(rid[0]) if rid else None
696 except Exception as e:
697     logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
698     {e}")
699     return None
700
701 def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
702     """Return the single active per-user model row for ``user_id`` (NULL = local
703     install),
704     with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
705     deserialised,
706     or None if the user has no active model. Does not touch the served pipeline."""
707     with self._connect() as conn:
708         if user_id is None:
709             row = conn.cursor().execute(
710                 "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
711                 1").fetchone()
712         else:
713             row = conn.cursor().execute(
714                 "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
715                 (user_id,)).fetchone()
716         if not row:
717             return None
718         d = dict(row)
719         for col in ("fusion_config", "classifier_json", "promotion_evidence"):
720             d[col] = json.loads(d[col]) if d[col] else None
721         return d

```

5. assistant (2026-06-21T09:25:01.224Z)

Now let me read the cost-attribution design doc, the API directory, and grep for auth code.

6. user (2026-06-21T09:25:02.215Z)

```
1      # 05 . Cost attribution ? per-video inference & per-run training billing
2
3      **Status:** PROPOSAL (net-new design) . **Date:** 2026-06-20
4
5      > **Side goal:** a non-owner user runs a job and gets the **exact billing for that
video** (inference), and
6      > training cost can be **segmented similarly** (per run, optionally amortized per
contributed video). This
7      > is wholly new ? the `jobs` table has **no cost columns** today
([`database.py:188`](../../backend/infrastructure/database.py)).
8      > The good news: a job is already the unit of work, and serverless platforms already
meter per invocation ?
9      > so per-video billing is mostly *bookkeeping on top of seams that exist.*
10
11     ## 1. The core idea
12
13     **A job is the billing unit.** An `infer` job is **1:1 with a video** (keyed by
`video_id`/MD5), so the
14     job's cost *is* the per-video bill. A `train` job is **1:1 with a training run**, so its
cost is the run
15     cost ? which we then optionally **split across the videos that fed it**.
16
17     ```
18     cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)
19     ```
20
21     Resources to meter: **GPU-seconds** (the big one), CPU-seconds, wall-seconds, **egress
GB** (reel
22     downloads ? the line `HOSTING_PLAN.md` flags), storage GB-hours, and **external-API
calls** (Gemini
23     tokens/calls ? the provider already returns metrics: `analyze_video` ? (segments,
metrics)).
24
25     ## 2. Metering mechanics (cheapest accurate source first)
26
27     | Source of GPU-seconds | When | Accuracy |
28     |---|---|---|
29     | **Serverless per-invocation report** (RunPod Serverless / Cloud Run / Modal) | M3
inference endpoint | **Best** ? the platform bills exact GPU-seconds per job; that number *is*
the bill. **This is the strongest argument for a serverless `infer` topology**
([03](03-PLATFORM-OPTIONS.md)). |
30     | **Worker-measured `gpu_active_seconds`** (timestamp around the GPU section, × the
known `gpu_type` rate) | DO Droplet / VM where there's no serverless meter | Good ? separate
`gpu_active_seconds` from `wall_seconds` so you don't bill the user for the bucket-sync/upload
time. |
31     | **Provider billing API reconciliation** (monthly) | all | Truth source ? reconcile the
sum of job costs against the invoice; carry a `reconciliation_factor`. |
32
33     Egress is counted from bytes downloaded (R2 = zero egress ? simplest). Gemini cost comes
from the metrics
34     the provider already returns.
35
36     ## 3. Schema delta (extends the existing `jobs` table; no new datastore)
37
38     Add via the existing `_add_column_idempotent` / `PRAGMA user_version` migration ladder:
39
40     ```sql
41     ALTER TABLE jobs ADD COLUMN owner_id          TEXT;      -- who pays (the non-owner user /
tenant)
42     ALTER TABLE jobs ADD COLUMN compute_backend TEXT;      -- runpod_serverless |
gcp_cloudrun | do_droplet ?
43     ALTER TABLE jobs ADD COLUMN gpu_type          TEXT;      -- L4 | A4000 | T4 ?
44     ALTER TABLE jobs ADD COLUMN gpu_active_seconds REAL;
```

```

45     ALTER TABLE jobs ADD COLUMN wall_seconds      REAL;
46     ALTER TABLE jobs ADD COLUMN bytes_out         INTEGER;    -- egress
47     ALTER TABLE jobs ADD COLUMN gemini_cost_usd    REAL;
48     ALTER TABLE jobs ADD COLUMN pricebook_version TEXT;
49     ALTER TABLE jobs ADD COLUMN cost_usd           REAL;        -- computed total
50     ALTER TABLE jobs ADD COLUMN cost_breakdown_json TEXT;      -- {gpu: .., egress: .., api:
..}
51
52     -- price book (versioned; rates change, and differ per provider/gpu_type)
53     CREATE TABLE IF NOT EXISTS pricebook (
54         version TEXT, resource TEXT, gpu_type TEXT, unit_price_usd REAL, currency TEXT,
effective_at TIMESTAMP
55     );
56
57     -- training-run cost attribution across contributing videos
58     CREATE TABLE IF NOT EXISTS training_contributions (
59         training_job_id TEXT, video_id TEXT, weight REAL    -- weight = clip-seconds share (or
1/N)
60     );
61     ``
62
63     The pricebook is seeded from the same provider-grounded GPU set as the device
profiles
64     ([07](07-COMPUTE-ABSTRACTION.md) §5) ? D0/Paperspace now (A4000 / L40S / A100), GCP
later (T4 / L4 / A100) ?
65     so one {provider, gpu_type} table carries both the $/hr rate (here) and the device-
tuning knobs (there),
66     keyed off the gpu_type that run_telemetry's nvidia-smi probe already yields.
67
68     owner_id is the per-tenant scoping the multi-tenant model
([PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §4)
69     will need anyway ? adding it now keeps "a non-owner user is billed for their jobs" a
first-class property.
70
71     ## 4. Per-video inference billing
72
73     1. An infer job's cost_usd + cost_breakdown_json is the per-video bill (job is
1:1 with the video).
74     2. Expose GET /api/jobs/{id}/cost and a per-video cost history (GET
/api/videos/{video_id}/cost), scoped
75     by owner_id.
76     3. Optional resale markup: a margin config ? price_to_user = cost_usd × (1 +
margin); store both so the
77     user sees a transparent breakdown and you keep your margin auditable.
78
79     Example breakdown (illustrative): 3-min L4 inference + a 40 MB reel download + one
Gemini call ?
80     gpu: 180 s × $0.00022/s = $0.040 + egress: 0.04 GB × $0 (R2) = $0 + gemini: ~$0.05
? ~$0.09/video.
81
82     ## 5. Per-training cost segmentation
83
84     1. A train job's cost_usd is the run cost (e.g. Gen-0 ? a few cents; a future
VLM run would be $-tens).
85     2. To "segment similarly," attribute the run cost across the N videos/labels in that run
via
86     training_contributions ? equal split (1/N) or weighted by clip-seconds
contributed. This
87     yields "cost per video contributed to training," the symmetric analog of the per-
video inference bill.
88     3. Record training_job_id on the produced weight bundle so a served model's lineage
carries its training
89     cost (useful for amortizing training into a per-inference price later, if you
choose).
90

```

```

91     ## 6. Trust & reconciliation
92
93     - Transparency: always show the user the `cost_breakdown_json` +
`pricebook_version`, not just a total.
94     - Reconciliation: metered cost ? provider invoice but not exact (overhead, rounding,
idle). Reconcile
95         monthly: `reconciliation_factor = invoice_total / ? metered_cost`; surface drift if it
exceeds a few %.
96     - Spend guard: a per-`owner_id` rate limit + a global hard cap (the same cap that
protects the $200)
97         means a leaked API key can't run up someone else's bill or drain the budget.
98
99     ## 7. Why this is cheap to build
100
101     It rides entirely on seams that exist: the `jobs` table (already the unit of work +
has `compute_backend`
102     in the design), the per-video output layout, the `tenant_id` the platform doc
already plans, and the
103     ComputeBackend (which knows `gpu_type` + rate). The only genuinely new pieces are
the pricebook, the
104     usage capture in the worker, and the attribution table ? all small, and none
requiring a new
105     datastore.
106

```

7. user (2026-06-21T09:25:03.625Z)

```

total 29
drwxr-xr-x 1 avidu 197609    0 Jun 19 13:03 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:07 ..
-rw-r--r-- 1 avidu 197609   59 Jun 18 15:52 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:48 __pycache__
-rw-r--r-- 1 avidu 197609 7780 Jun 19 13:03 job_worker.py
drwxr-xr-x 1 avidu 197609    0 Jun 11 20:44 static
-rw-r--r-- 1 avidu 197609 6882 Jun 18 15:52 uploads.py
---STORAGE---
total 52
drwxr-xr-x 1 avidu 197609    0 Jun 21 13:37 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:07 ..
-rw-r--r-- 1 avidu 197609 6326 Jun 21 13:37 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:48 __pycache__
-rw-r--r-- 1 avidu 197609 2231 Jun 20 15:34 base.py
-rw-r--r-- 1 avidu 197609 2661 Jun 20 13:48 gcs.py
-rw-r--r-- 1 avidu 197609 6910 Jun 20 15:47 gdrive.py
-rw-r--r-- 1 avidu 197609 2756 Jun 20 13:48 local.py
-rw-r--r-- 1 avidu 197609 4455 Jun 21 13:23 ref.py
-rw-r--r-- 1 avidu 197609 3728 Jun 20 13:48 s3.py

```

8. assistant (2026-06-21T09:25:06.567Z)

Now let me look at the API layer, the storage registry/base, and grep for any auth/JWT/Cf-Access code, plus find the project doc and where the FastAPI app + routes live.

9. user (2026-06-21T09:25:08.623Z)

```

Found 2 files
backend\main.py
artifacts\so.py

```

10. user (2026-06-21T09:25:09.423Z)

```

1     """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3     Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket

```

```

backends
4     live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5     pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6     no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7     """
8     from __future__ import annotations
9
10    import os
11    from abc import ABC, abstractmethod
12    from typing import Iterator
13
14    from backend.storage.ref import StorageRef, StorageStat
15
16
17    class StorageBackend(ABC):
18        """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19        worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21        @abstractmethod
22        def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                        *, overwrite: bool = False) -> str:
24            """Make the object available as a local file; return its local path. For the
25            ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27        @abstractmethod
28        def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29               content_type: "str | None" = None) -> StorageRef:
30            """Upload/copy a local file to ``ref``; return ``ref``."""
31
32        @abstractmethod
33        def exists(self, ref: StorageRef) -> bool: ...
34
35        @abstractmethod
36        def stat(self, ref: StorageRef) -> StorageStat: ...
37
38        @abstractmethod
39        def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40            """Yield a ``StorageRef`` per object under ``prefix``."""
41
42        @abstractmethod
43        def delete(self, ref: StorageRef) -> None: ...
44
45        def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46            """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47            sign (``local`` cannot; Drive only emulates) ? callers must treat
48            ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49            raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50

```

11. user (2026-06-21T09:25:09.948Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive`` likewise needs none ? it reads a locally
MOUNTED

```

```

7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13    from __future__ import annotations
14
15    import os
16    import tempfile
17    from typing import Any, Dict, Iterator, Optional
18
19    from backend.storage.base import StorageBackend
20    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22    __all__ = [
23        "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24        "get_storage", "fetch", "put", "list_uris",
25        "acquire_local_input", "input_is_local", "resolve_input_ref",
26    ]
27
28
29    def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30        """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31        if storage_config is None:
32            return {}
33        if hasattr(storage_config, "model_dump"):
34            return storage_config.model_dump() # type: ignore[no-any-return]
35        return dict(storage_config)
36
37
38    def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39        """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40        config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41        callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43        Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44        ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45        sc = _storage_dict(storage_config)
46        uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                               input_root=sc.get("input_root", ""))
48        return parse_uri(uri)
49
50
51    def input_is_local(raw: str, storage_config: Any = None) -> bool:
52        """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53        plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54        that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55        ``resolve_input_ref``)."""
56        return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59    def acquire_local_input(raw: str, storage_config: Any = None, *,
60                           scratch_parent: "str | None" = None) -> str:
61        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62

```

```

63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (`gs://`/`s3://`/`gdrive://`, or a
bare name under a
65         non-local `input_root`/`input_backend`) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68         worker's `cmd_infer` already ships."""
69         sc = _storage_dict(storage_config)
70         ref = resolve_input_ref(raw, sc)
71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. `backend` (or `config['backend']`, default 'local')
selects
97         the implementation; its per-backend settings come from `config[backend]` merged
with
98         `overrides`. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve `uri` to a local file path. For `local` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to `dst`. """
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->

```

```

str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127

```

12. assistant (2026-06-21T09:25:13.261Z)

Auth/JWT terms only appear in `backend/main.py` and `artifacts/so.py` (the latter is scratch). Let me check `main.py` for the actual FastAPI app, CORS, and route definitions, plus the project doc README and the gemini provider metrics signature.

13. user (2026-06-21T09:25:14.601Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
33    from backend.results import Failure # standardized error/result contract
34    from backend.reporting import emit_indexer_report
35    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
36
37    app = FastAPI(title="Sports Highlight Indexer API")

```

```

38
39 # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
40 # rejected by browsers per the fetch spec and is an unsafe default to carry into the
41 # auth/tenancy work; this tool uses no cookies, so credentials stay off.
42 app.add_middleware(
43     CORSMiddleware,
44     allow_origins=["*"],
45     allow_credentials=False,
46     allow_methods=["*"],
47     allow_headers=["*"],
48 )
49
50 # Serves static frontend files directly (now under api/ per approved structure)
51 os.makedirs("backend/api/static", exist_ok=True)
52 app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
53
54 @app.get("/", response_class=HTMLResponse)
55 def serve_index():
56     index_path = "backend/api/static/index.html"
57     if os.path.exists(index_path):
58         with open(index_path, "r", encoding="utf-8") as f:
59             return f.read()
60     return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
61
62 # Global variables initialized at startup
63 db_instance: Optional[Database] = None
64 global_config: Optional[AppConfig] = None # now typed (Pydantic model)
65
66 # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
67 # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
68 # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
69 # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
70 # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
71 # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
72 # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
73 from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
74     JobWaiting,
75     _arm_wake_timer,
76     _drain_due_jobs,
77     _get_executor,
78     _job_to_request,
79     _MAX_DRAIN_WAKE_SEC,
80     _run_claimed_job,
81     _schedule_next_drain_if_waiting,
82 )
83
84 # Legacy thin wrapper for any remaining direct calls during transition.
85 # New code should import load_app_config / AppConfig from backend.config directly.
86 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
87     cfg = load_app_config(config_path)
88     return cfg.model_dump(mode="json")
89
90 # --- API Models ---
91 class ProcessRequest(BaseModel):
92     video_path: str
93     player_pool: Optional[List[str]] = None
94     max_frames: Optional[int] = None
95     segmenter: Optional[str] = None
96
97 class QueryRequest(BaseModel):

```



```

98     video_id: str
99     server: Optional[str] = None
100    receiver: Optional[str] = None
101    duration_min: Optional[float] = None
102    event_type: Optional[str] = None
103
104    class CompileRequest(BaseModel):
105        video_id: str
106        segment_ids: List[int]
107        output_filename: str
108
109    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
110        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
111
112        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
113        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
114        and
115        keep the prior good results intact ? a failed/empty re-run never destroys prior
116        data.
117        """
118        if segments:
119            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
120        else:
121            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
122
123    # --- API Endpoints ---
124    @app.get("/api/config")
125    def get_config():
126        return global_config
127
128    @app.get("/api/health")
129    def health():
130        """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
131        (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
132        confirm the engine is reachable and which detector is wired. Never raises."""
133        sport = getattr(global_config, "sport", None)
134        indexing = getattr(global_config, "indexing", None)
135        return {
136            "status": "ok",
137            "sport": sport,
138            "default_segmenter": getattr(indexing, "default_segmenter", None),
139        }
140
141    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
142        """The full hash ? validate ? segment ? report pipeline for one video.
143
144        This is the former synchronous /api/process body, unchanged in behavior, now run on
145        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
146        the sync endpoint used to return (UI compatibility); the per-video observability
147        report is still emitted in `finally:` and grafted as response["reports"].
148
149        `progress` is an optional callable(stage: str) used to surface coarse job progress.
150        Raises on unexpected internal errors ? the caller records those as a job failure
151        (after this function has already restored the pre-run segments, see below).
152        """
153        notify = progress or (lambda stage: None)
154
155        notify("hashing")
156        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
157        (identity ?
158         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
159         original
160         # req.video_path (the source URI) is kept for the DB record + report; only the file-
161         reading

```

```

158         # consumers (hash / validators / segmenter) use the resolved local path.
159         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
160         video_id = compute_video_id(local_video)
161         was_reingest = db_instance.get_video(video_id) is not None
162
163         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
164         notify("validating")
165         logger.info(f"Running validations for {req.video_path}...")
166         val_results, val_context = registry.run_all_with_context(local_video, global_config)
167         failures = [r for r in val_results if not r.passed]
168
169         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
170         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
171         seg_name = req.segmenter or default_seg
172         segmenter_actual = None
173         segmentation_ran = False
174         response: Optional[Dict[str, Any]] = None
175         try:
176             if failures:
177                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
178                 # status; it no longer destroys the video's prior good segments.
179                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
180                 response = {
181                     "video_id": video_id,
182                     "status": "failed",
183                     "message": "Video failed validation checks.",
184                     "results": [r.model_dump() for r in val_results],
185                 }
186                 return response
187
188             # 2. Run segmenter & indexer
189             logger.info("[API] Video passed checks. Segmenting and indexing...")
190             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
191
192             segmenter_cls = segmenter_registry.get(seg_name)
193             if not segmenter_cls:
194                 # Submit-time validation already 400s unknown names; this is the defensive
195                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
196                 available = list(segmenter_registry.list_available().keys())
197                 response = {
198                     "video_id": video_id,
199                     "status": "failed",
200                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
201                     "results": [r.model_dump() for r in val_results],
202                     "play_segments": [],
203                 }
204                 return response
205
206             logger.info(f"[API] Using segmenter: {seg_name}")
207             notify(f"segmenting ({seg_name})")
208             segmenter_actual = seg_name
209             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
210             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
211             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
212             segmenter = segmenter_cls(db_instance, global_config)
213             try:

```

```

214         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
215     except Exception:
216         # A raising segmenter must not leave half-written rows: restore the pre-run
217         # state (same destroy-after-confirm contract as the Failure branch), then
218         let
219             # the job worker record the failure.
220             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
221             raise
222     segmentation_ran = True
223
224     if isinstance(result, Failure):
225         logger.error(f"[API] Segmenter failed: {result.message}")
226         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
227         response = {
228             "video_id": video_id,
229             "status": "failed",
230             "message": f"Segmenter '{seg_name}' failed: {result.message}",
231             "results": [r.model_dump() for r in val_results],
232             "play_segments": [],
233         }
234         return response
235
236     segments = result.value if hasattr(result, "value") else result
237     notify("finalizing")
238     _finalize_reingest(video_id, segments, play_wm, cand_wm)
239
240     response = {
241         "video_id": video_id,
242         "status": "success",
243         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
244         "results": [r.model_dump() for r in val_results],
245         "play_segments": segments,
246     }
247     return response
248 finally:
249     # Always emit the per-video observability report (next to the video, or to
250     # report.output_dir). Never raises. Surface the paths in the response.
251     paths = emit_indexer_report(
252         db=db_instance, video_id=video_id, video_path=req.video_path,
253         config=global_config,
254         val_results=val_results, val_context=val_context,
255         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
256         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
257     )
258     if paths and isinstance(response, dict):
259         response["reports"] = paths
260
261 @app.post("/api/process", status_code=202)
262 def process_video(req: ProcessRequest):
263     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
264
265     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
266     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
267     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
268     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
269     """
270     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
271     try:
272         validate_input_path(req.video_path, allowed_root=allowed_root)
273         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
274         unsupported

```

```

273         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
274         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
275     except ValueError as e:
276         raise HTTPException(status_code=400, detail=str(e)) from e
277     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
278     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
279     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
280     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
281     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
282     # rejects a non-local URI as out-of-root.
283     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
284         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
285     if req.segmenter and not segmenter_registry.get(req.segmenter):
286         available = list(segmenter_registry.list_available().keys())
287         raise HTTPException(
288             status_code=400,
289             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
290         )
291
292     job_id = uuid.uuid4().hex
293     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
294                                req.player_pool, req.max_frames):
295         raise HTTPException(status_code=500, detail="Failed to persist job record.")
296     _get_executor().submit(_drain_due_jobs)
297     return JSONResponse(
298         status_code=202,
299         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
300     )
301
302
303 @app.get("/api/jobs/{job_id}")
304 def get_job(job_id: str):
305     """Job state: {status, progress, result, reports}. `status` = execution state
306     (queued|running|done|failed); the pipeline verdict is result["status"]."""
307     job = db_instance.get_job(job_id)
308     if not job:
309         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
310     result = job.get("result")
311     return {
312         "job_id": job["id"],
313         "status": job["status"],
314         "progress": job.get("progress"),
315         "video_id": job.get("video_id"),
316         "error": job.get("error"),
317         "result": result,
318         "reports": (result or {}).get("reports"),
319         "created_at": job.get("created_at"),
320         "started_at": job.get("started_at"),
321         "finished_at": job.get("finished_at"),
322     }
323
324 # --- Chunked upload (B2, PR-2) -----
325 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
326 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
327 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,

```

```

328     # sequential-part logic, and abort cleanup ? is unchanged.
329     from backend.api import uploads as uploads_api
330
331     app.include_router(uploads_api.router)
332
333
334     # --- Highlight download (B3, PR-2) -----
335
336     @app.get("/api/highlights/{video_id}/{filename}")
337     def download_highlight(video_id: str, filename: str):
338         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
339         (which only writes into output_dir; the browser previously got back an unreachable
340         server-local path)."""
341         try:
342             name = safe_output_filename(filename)
343             except ValueError as e:
344                 raise HTTPException(status_code=400, detail=str(e)) from e
345             if not db_instance.get_video(video_id):
346                 raise HTTPException(status_code=404, detail="Indexed video record not found.")
347             output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
348             path = os.path.join(output_dir, name)
349             try:
350                 path = resolve_under_root(path, output_dir)
351             except ValueError as e:
352                 raise HTTPException(status_code=400, detail=str(e)) from e
353             if not os.path.isfile(path):
354                 raise HTTPException(status_code=404,
355                                     detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
356             media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
357             return FileResponse(path, media_type=media, filename=name)
358
359
360     @app.post("/api/query")
361     def query_play_segments(req: QueryRequest):
362         filters = {
363             "server": req.server,
364             "receiver": req.receiver,
365             "duration_min": req.duration_min,
366             "event_type": req.event_type
367         }
368         segments = db_instance.query_play_segments(req.video_id, filters)
369         return {"play_segments": segments}
370
371     @app.post("/api/compile")
372     def compile_highlights(req: CompileRequest):
373         video = db_instance.get_video(req.video_id)
374         if not video:
375             raise HTTPException(status_code=404, detail="Indexed video record not found.")
376
377         # Retrieve matching play segments from db
378         all_segments = db_instance.query_play_segments(req.video_id, {})
379         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
380
381         if not matched_segments:
382             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
383
384         # Reject path traversal / absolute output names (os.path.join would otherwise let an
385         # absolute or '..'-laden output_filename write anywhere on disk).
386         try:
387             out_name = safe_output_filename(req.output_filename)
388             except ValueError as e:

```

```

389         raise HTTPException(status_code=400, detail=str(e)) from e
390
391     # The configured shot-count floor (stitching.min_shot_count) applies on the API
392     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
393     # metadata are exempt: the floor filters known counts only, so explicitly
394     # requested manual/legacy segments can't silently vanish under the default floor.
395     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
396     kept = []
397     for s in matched_segments:
398         meta = s.get("metadata") or {}
399         sc = meta.get("shot_count") if isinstance(meta, dict) else None
400         try:
401             if sc is not None and int(sc) < stitching.min_shot_count:
402                 continue
403             except (TypeError, ValueError):
404                 pass # unparseable count -> treat as unknown, keep
405             kept.append(s)
406         if len(kept) < len(matched_segments):
407             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
408                         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
409             if not kept:
410                 raise HTTPException(status_code=400,
411                                     detail=f"All {len(matched_segments)} matching segments fall
below "
412                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
413
414     # Extract start/end time pairs
415     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
416
417     # Run compiler with the configured stitching paddings + optional branding watermark
418     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
419     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
420                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
421     try:
422         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
423         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
424         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
425         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
426         return {"status": "success", "filepath": out_path, "download_url": download_url}
427     except Exception as e:
428         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
429
430     # --- CLI Debug Utility & Orchestration ---
431     def run_cli_diagnose_clip(video_path: str, timestamp: float):
432         """CLI debugging utility to examine segment properties around a timestamp."""
433         print("=" * 60)
434         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
435         print("=" * 60)
436
437     cfg = load_config() # legacy wrapper still works, returns dict for now
438
439     # 1. Validation check diagnostics
440     print("[DIAGNOSTIC] Running validation framework...")
441     results = registry.run_all(video_path, cfg)
442     for r in results:
443         status_symbol = "[PASS]" if r.passed else "[FAIL]"

```

```

444         print(f" {status_symbol} {r.validator_name}: {r.message}")
445     if r.details:
446         print(f"         Details: {r.details}")
447
448     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
449     print("-" * 60)
450     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
451     import cv2
452     cap = cv2.VideoCapture(video_path)
453     if not cap.isOpened():
454         print("[FAIL] OpenCV could not open video.")
455         return
456
457     fps = cap.get(cv2.CAP_PROP_FPS)
458     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
459
460     start_frame = max(0, int((timestamp - 3.0) * fps))
461     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
462
463     # Measure frame-by-frame changes in sample region
464     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
465     prev_gray = None
466     motions = []
467
468     for _ in range(start_frame, end_frame):
469         ret, frame = cap.read()
470         if not ret:
471             break
472         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
473         gray = cv2.GaussianBlur(gray, (21, 21), 0)
474
475         if prev_gray is not None:
476             diff = cv2.absdiff(prev_gray, gray)
477             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
478             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
479             motions.append(motion_ratio)
480             prev_gray = gray
481
482     cap.release()
483
484     if motions:
485         avg_motion = sum(motions) / len(motions)
486         # Support both legacy dict (from wrapper) and future direct model
487         if hasattr(cfg, "indexing"):
488             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
489         else:
490             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
491         print(f" Sample Frame Count: {len(motions)}")
492         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
493         if avg_motion > threshold:
494             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
495         else:
496             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
497     else:
498         print(" [ERROR] No visual frames were extracted in the sample range.")
499
500     print("=" * 60)
501
502 def main():
503     parser = argparse.ArgumentParser(description="Sports Highlight Indexer

```

```

Orchestrator")
504     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
505     parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
506     parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
507     parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
508     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
509     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
510     parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
511
512     available_segmenters = list(segmenter_registry.list_available().keys())
513     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
514     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
515     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
516     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
517
518     args = parser.parse_args()
519
520     if args.setup:
521         # Invoke the diagnostics/bootstrap script (renamed from the old root
522         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
523         # build system). Resolve its path from this file so `indexer --setup`
524         # works regardless of the caller's cwd.
525         import subprocess
526         doctor_path = os.path.join(
527             os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
528             "scripts", "doctor.py",
529         )
530         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
531         subprocess.run([sys.executable, doctor_path])
532         return
533
534     if args.trim_start is not None and args.trim_end is not None:
535         if not args.video_path:
536             print("[ERROR] Please provide --video-path to extract a segment.")
537             return
538
539         # Determine output path
540         out_filename = args.trim_output or "trimmed_segment.mp4"
541         if os.path.isabs(out_filename):
542             out_path = out_filename
543             out_dir = os.path.dirname(out_path)
544             out_name = os.path.basename(out_path)
545         else:
546             out_dir = args.output_dir
547             out_path = os.path.join(out_dir, out_filename)
548             out_name = out_filename
549
550         os.makedirs(out_dir, exist_ok=True)
551         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
552
553         # global_config isn't initialized this early; load the typed config so the trim
554         # clip carries the same configured stitching paddings + watermark as a real
555         # highlight (watermark default-on).
556         stitching = load_app_config().stitching

```



```

557         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
558                                   end_padding=stitching.end_padding,
watermark=stitching.watermark)
559         try:
560             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
561             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
562         except Exception as e:
563             print(f"[ERROR] Failed to compile trimmed segment: {e}")
564         return
565
566     if args.debug_clip is not None:
567         if not args.video_path:
568             print("[ERROR] Please provide --video-path to debug a specific clip.")
569             return
570         run_cli_diagnose_clip(args.video_path, args.debug_clip)
571         return
572
573     # Initialize Global Config and DB
574     global global_config, db_instance
575     global_config = load_app_config() # returns typed AppConfig
576
577     # The CLI --output-dir overrides the configured output directory. It now lives
578     # on the typed model (OutputConfig.output_dir), so every consumer ? including
579     # /api/compile ? reads the same value instead of a write-only runtime hack.
580     global_config.output.output_dir = args.output_dir
581     os.makedirs(global_config.output.output_dir, exist_ok=True)
582
583     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
584     db_instance = Database(db_path)
585
586     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
587     # the executor is in-process. Mark them failed so pollers see a terminal state.
588     swept = db_instance.fail_stale_jobs()
589     if swept:
590         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
591
592     # CLI processing mode (no web UI needed)
593     if args.video_path and not args.server:
594         print(f"[CLI] Processing video file: {args.video_path}")
595         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
596         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
597         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
598         # not a crash.
599         storage_cfg = getattr(global_config, "storage", None)
600         try:
601             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
602         except ValueError as e:
603             print(f"[FAIL] {e}")
604             return
605         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
606             print(f"[FAIL] Video file not found: {args.video_path}")
607             return
608         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
609
610         video_id = compute_video_id(local_video)
611         was_reingest = db_instance.get_video(video_id) is not None
612
613         # Validate (with-context variant surfaces ffprobe_meta for the report)
614         print(f"[CLI] Validating...")
615         val_results, val_context = registry.run_all_with_context(local_video,

```

```

global_config)
615         failures = [r for r in val_results if not r.passed]
616
617         # Save video status
618         status = "failed" if failures else "processed"
619         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
620
621         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
622         segmenter_actual = None
623         segmentation_ran = False
624         try:
625             print("\n" + "="*40)
626             print("VALIDATION SUMMARY")
627             print("="*40)
628             for r in val_results:
629                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
630                 print(f"{status_symbol} {r.validator_name}: {r.message}")
631             print("="*40 + "\n")
632
633             if failures:
634                 print("[FAIL] Video failed checks. Indexing halted.")
635                 return
636
637             # Index
638             segmenter_cls = segmenter_registry.get(seg_name)
639             if not segmenter_cls:
640                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
641                 return
642
643             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
644             segmenter_actual = seg_name
645             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
646             segmenter = segmenter_cls(db_instance, global_config)
647             result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
648             segmentation_ran = True
649
650             if isinstance(result, Failure):
651                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
652                 print("[FAIL] Indexing halted for video.")
653                 segments = []
654             else:
655                 segments = result.value if hasattr(result, "value") else result
656                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
657                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
658                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
659             finally:
660                 # Always emit the per-video observability report. Never raises.
661                 paths = emit_indexer_report(
662                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
663                     val_results=val_results, val_context=val_context,
664                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
665                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
666                 )
667             if paths:
668                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
669                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
670                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
671         return
672

```

```

673         # Start Local Server Mode
674         if args.server or not args.video_path:
675             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
676             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
677             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
678             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
679             _get_executor().submit(_drain_due_jobs)
680             uvicorn.run(app, host="127.0.0.1", port=args.port)
681
682     if __name__ == "__main__":
683         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
684         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
685         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
686         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
687         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
688         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
689         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
690         import backend.main as _bm
691
692         _bm.main()
693

```

14. user (2026-06-21T09:25:14.678Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...) ` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18    import logging
19    import threading
20    import traceback
21    from concurrent.futures import ThreadPoolExecutor
22    from typing import Any, Dict, Optional
23
24    logger = logging.getLogger(__name__)
25
26    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29    # already parallelize their AI handoff internally via their own pools.
30    _job_executor: Optional[ThreadPoolExecutor] = None

```

```

31
32
33     def _get_executor() -> ThreadPoolExecutor:
34         """Lazy module-global executor; tests monkeypatch this for inline execution."""
35         global _job_executor
36         if _job_executor is None:
37             _job_executor = ThreadPoolExecutor(max_workers=1,
38 thread_name_prefix="jobworker")
39         return _job_executor
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
45         `set_job_waiting`),
46         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
47         date
48         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
49         contract.
50
51         The wiring is additive: the production segmenter path never raises this today (zero
52         behaviour change), so a job runs exactly as before. The hook is what a later slice
53         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
54         """
55
56         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
57             super().__init__(f"job parked for {delay_sec:.0f}s")
58             self.delay_sec = max(0.0, float(delay_sec))
59             self.progress = progress
60
61     def _job_to_request(job: Dict[str, Any]):
62         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
63         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
64         the original submit. The row stores exactly the ProcessRequest fields."""
65         import backend.main as _main
66         return _main.ProcessRequest(
67             video_path=job["video_path"],
68             player_pool=job.get("player_pool"),
69             max_frames=job.get("max_frames"),
70             segmenter=job.get("segmenter"),
71         )
72
73     def _run_claimed_job(job: Dict[str, Any]) -> None:
74         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
75         its
76         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
77
78         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
79         means
80         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
81         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
82         the
83         job self-scheduled and will be re-claimed when `next_poll_at` is due.
84         """
85         import backend.main as _main
86         db_instance = _main.db_instance
87         job_id = job["id"]
88         req = _main._job_to_request(job)
89         try:
90             result = _main._execute_processing(
91                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
92             if result.get("video_id"):

```

```

89         db_instance.set_job_video_id(job_id, result["video_id"])
90         db_instance.mark_job_done(job_id, result)
91     except _main.JobWaiting as w:
92         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
93         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
94         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
95     except Exception as e:
96         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
97         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
98
99
100     def _drain_due_jobs() -> int:
101         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
102         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
103
104         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
105         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
106         a single-worker box keeps making progress with no central timer (the DB is the
clock).
107         Returns the number of jobs that reached a terminal/parked state this tick.
108         """
109         import backend.main as _main
110         ran = 0
111         while True:
112             job = _main.db_instance.claim_due_job()
113             if job is None:
114                 break
115             ran += 1
116             _main._run_claimed_job(job)
117         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
118         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
119         # on the next submit/drain. Best-effort ? never raises into the worker.
120         _main._schedule_next_drain_if_waiting()
121         return ran
122
123
124     def _schedule_next_drain_if_waiting() -> None:
125         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
126         self-scheduled job resumes with no further client submit. The drain itself re-checks
127         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128         import backend.main as _main
129         try:
130             delay = _main.db_instance.seconds_until_next_due()
131         except Exception as e: # pragma: no cover - defensive; never wedge the worker
132             logger.error(f"Could not compute next-due delay: {e}")
133             return
134         if delay is None:
135             return # nothing waiting
136         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140     def _arm_wake_timer(delay_sec: float) -> None:
141         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143         monkeypatch it to a no-op and assert the *scheduling decision* without real

```

```

waiting."""
144     import backend.main as _main
145     timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146     timer.daemon = True
147     timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153

```

15. assistant (2026-06-21T09:25:21.917Z)

Now I have a clear picture of the API/auth/CORS reality. Let me look at the project doc README to confirm milestone definitions (M8/M9/M12), the gemini provider metrics signature, and config schema for any deployment/security fields. Let me also check the s3/gcs backends to understand the signed_url pattern that an OAuth gdrive backend would mirror.

16. user (2026-06-21T09:25:23.336Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta
11    from typing import Any, Iterator, Optional
12
13    from backend.storage.base import StorageBackend
14    from backend.storage.ref import StorageRef, StorageStat
15
16
17    class GCSStorage(StorageBackend):
18        def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                    signed_url_ttl: int = 3600, **_ignored) -> None:
20            self._ttl = signed_url_ttl
21            if client is not None:
22                self._client = client
23            return
24            from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25                self._client = storage.Client(project=project)
26
27        def _blob(self, ref: StorageRef):
28            return self._client.bucket(ref.bucket).blob(ref.key)
29
30        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31            if dst is None:
32                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33            import os
34            dst = str(dst)
35            if os.path.exists(dst) and not overwrite:
36                return dst
37            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38            self._blob(ref).download_to_filename(dst)

```

```

39         return dst
40
41     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43         return ref
44
45     def exists(self, ref) -> bool:
46         return bool(self._blob(ref).exists())
47
48     def stat(self, ref) -> StorageStat:
49         b = self._blob(ref)
50         b.reload()
51         return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
57 f"gcs://{prefix.bucket}/{blob.name}")
58
59     def delete(self, ref) -> None:
60         self._blob(ref).delete()
61
62     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
63         return self._blob(ref).generate_signed_url(
64             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

17. user (2026-06-21T09:25:23.363Z)

```

1     # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3     > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .
**Owner:** Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5     > **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6     > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7     > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8     > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10    ---
11
12    > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
**to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect **suggests**, the user
**confirms** (D15).
13
14    ## 0. TL;DR
15    The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch
**on**. A **`ComputeBackend` (deployment profile)** decides **where** the core runs (a local process
vs a Cloud Run GPU job); a **`StorageBackend`** decides **where bytes come from** (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus

```

****cpu-mock**** for CI. The single most important caveat: detection is ****byte-identical** only on the same GPU¹⁶ (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the ****parity guarantee** is enforced at the WINDOW + STITCH layer¹⁷, not detector-CSV bytes.

16
17 **## 1. Problem & goal**
18 - ****Why now:**** DECOUPLED_COMPUTE proved compute ***portability*** (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred ****M3** (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)¹⁸ become this project. The owner needs ****both**** a hosted service ***and*** a truly-local mode, with the ****core** (detect+stitch) unaffected¹⁹ by which one runs.
19 - ****The two user-facing outcomes**** (owner, 2026-06-21):
20 - ****a) Hosted service**** ? operate on small ****local uploads (<2GB)**** ***and*** load from ****gdrive / a bucket****, spinning up ****Cloud Run** for both rally segmentation AND stitching²⁰ (stitching is compute-intensive ? run it where its cost is accounted for).
21 - ****b) Truly-local**** ? on a local machine with videos on ****local drives/USB + a local GPU****, run inference ****and**** stitching ****completely locally****.
22 - ****"Good" looks like:**** the same **`video ? rallies ? reel`** core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a ****config/startup choice****, never a code branch in the core.

23
24 **## 2. Decisions locked**
25
26 | # | Decision | Source | Implication |
27 |---|-----|-----|-----|
28 | D1 | The core is a ****pure 3-stage function**** (DETECT/WINDOW/STITCH); ****no profile branch inside it****. | design **`[verified]`** | All profile difference lives in config + the two registries. |
29 | D2 | ****Two registries:**** **`ComputeBackend`** (where) selected by **`deployment.profile`** + **`compute\_target`**; **`StorageBackend`** (what-bytes) by **`input\_backend`/`storage.backend`**. | ADR-014 | Realizes PLATFORM_ARCHITECTURE's **`local`/`hosted`/`byo\_worker`**. |
30 | D3 | ****Profiles shipped:**** **`truly-local`**, **`cloud-serving`**, **`cpu-mock`** (CI); **`byo\_worker`** reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31 | D4 | In ****cloud-serving**, STITCH runs on Cloud Run³¹ (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real cost. |
32 | D5 | ****Parity enforced at WINDOW+STITCH**** (byte-exact for a fixed trajectory); ****detect**** is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the rally/window level, never CSV bytes. |
33 | D6 | ****Default-OFF, smallest-safe-first, one PR per milestone****; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report. | [[launch-report-convention]] | Zero-regression discipline. |
34 | D7 | ****Cloud Run cold-start: SCALE-TO-ZERO**** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). ****No warm pool**** (that ~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |
35 | D8 | ****Pricebook = a versioned DB table****; cost computed in ****USD**** (matches GCP billing = one source of truth), ****displayed in INR**** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |
36 | D9 | ****Edge auth = Cloudflare Access**** (reuse the site's existing CF). Lock the Cloud Run ****ingress to the CF proxy**** + ****verify the `Cf-Access` JWT signature**** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |
37 | D10 | ****Free-tier "data-for-reels" trade:**** free reels in exchange for ****training rights**** (uploaded footage + derived trajectories/labels); ****paid tier = no-train**** (privacy is the paid feature). ****Carve-outs:**** train ONLY on attested ****ADULT**** footage (minor footage ? reel yes, training pool ****NO**** ? DPDP/GDPR need verifiable parental consent); train on ****DERIVED**** data + ****auto-delete raw****; ****ToS counsel-reviewed****; live training-on-uploads ****gated to M9**** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel. |
38 | D11 | ****Quality floor: Gemini handoff ON**** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a ****Launch Report****; the owned model runs in ****shadow/eval**** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39 | D12 | ****Data-flywheel harness is IN SCOPE** (owner add, Q5):³⁹ capture the (consented, adult) uploaded video + the Gemini reel/rally output ? ****reliably store**** in the per-video workspace/bucket ? ****run shadow evals**** (owned-vs-Gemini, dual-eval-set) ? ****promote good ones**

to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new **M11** ; ties D10 (consent) + `data\_pipeline/` + the eval/experiment harness. |

40 | D13 | **Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is *between* jobs. |

41 | D14 | **NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. | owner 2026-06-21 | Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves S8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |

42 | D15 | **Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently** ; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |

43

44 ## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*

45 This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:

46

47 | ADR / doc | Contributed | Subtlety carried into this project |

48 |---|---|---|

49 | **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |

50 | **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**. |

51 | **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class. |

52 | **ADR-010** | DECOUPLED: D0?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |

53 | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the "rent nothing / not a service" posture | **This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**. |

54 | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model** ; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |

55 | **ADR-013** | Drop D0; **GCP sole compute stack** ; \$200 D0 credits ? non-compute only | The provider for cloud-serving; D0 Spaces remains only a possible S3 *storage* target. |

56 | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not** supersede them. |

57 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |

58 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |

59 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |

60 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel parity finding** | Why D5 (parity at window level, not CSV bytes). |

61

62 ## 4. Design / architecture

63 See [`architecture.svg`](architecture.svg) for the picture. **The core never changes; the box around it does.**

64

65 ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`

66 1. **DETECT (GPU):** `DetectorRunner.run\_predict(video\_path, output\_dir, video\_id) ? CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable

impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.

67 2. **WINDOW** (CPU pure fn): **trajectory_to_action_windows**(points, fps, frame_width, ?) ? windows` (`tracknet_runner.py:281`). A shared **@staticmethod**, zero GPU/IO/randomness ? same trajectory + config = byte-identical candidate windows. Used by every runner **and** the eval/regression stack verbatim.

68 3. **STITCH** (CPU/ffmpeg): **VideoCompiler.compile_highlights**(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffmpeg ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.

69

70 **Non-goal** (the trap to avoid): **no** code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).

71

72 **### 4.2 The deployment profiles**

73 | Profile | Compute | Storage | Orchestration | When |

74 |---|---|---|---|---|

75 | **truly-local** | **compute_target=local_gpu**; **detector_impl=native** (Linux) / **auto** (WSL); **stitch in-process** | **local** (or **gdrive** = mounted Drive) | **python -m backend.main** or **worker infer** CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |

76 | **cloud-serving** | **compute_target=cloud_run**; **detector_impl=native** (L4, cuda); **detect AND stitch** on the same Cloud Run job | **gcs** (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; **WAIT_AND_RESUME** reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |

77 | **cpu-mock** | **compute_target=cpu_mock**; **detector_impl=stub** (synth/replay) | **local/in-memory fakes** | **run_all_tests.py** / **pytest** | CI, regression, profile-parity, GPU-free dev |

78 | **byo_worker** (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |

79

80 **### 4.3 Profile selection** (the "startup/setup config") **[design]**

81 ONE new top-level **deployment** section on **AppConfig** + preset application + env auto-detect in **load_config**:

82 - **deployment.profile** ? {truly-local | cloud-serving | cpu-mock} (empty = today's manual knobs, **fully backward-compatible**).

83 - Selecting a profile applies a **coherent preset bundle** of **existing** knobs that today drift independently: **INPUT** (new **input_backend**/**input_root**, parallel to the output-side **StorageConfig**), **COMPUTE** (new **compute_target** enum locking **detector_impl** + **skip_ai_handoff** + workspace strategy), **STORAGE** (**storage.backend**/**workspace_root**/**single_folder_per_video**, all already exist).

84 - **Precedence**: built-in defaults ? profile preset ? **config.json** ? env (**RALLY_DETECTOR_IMPL**, **WASB_***, **DEPLOYMENT_PROFILE**, ?) ? worker **--set k=v**. Every knob stays individually overridable.

85 - **Env auto-detect SUGGESTS**, the user **CONFIRMS** (D15 ? no surprise execution): the env gives a **proposed** default (**K_SERVICE** ? cloud-serving; **CI=true** ? cpu-mock; else **torch.cuda.is_available()** ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend)** is **NEVER** entered silently. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).

86 - The worker (**cmd_infer**/**cmd_train**) is **already profile-agnostic** (reads **config.storage**, accepts **--config**/**--set**) ? no worker rewrite.

87

88 **### 4.4 Compute placement** ? and **why stitch runs on Cloud Run** **[design]**

89 **DETECT** on GPU everywhere; **WINDOW** CPU-pure everywhere; **STITCH** is real CPU work. In **cloud-serving**, stitch is co-located in the Cloud Run job so **gpu_active_seconds** (detect) + **wall_seconds** (detect+stitch) + **bytes_out** (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + **claim_due_job** already supports both.)

90

91 **### 4.5 Reuse map** ? most of the seams already exist **[verified]**

```

92     **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam);
`StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends +
`StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control
plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting`
(carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff`
(LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video
workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns
exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill
hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-
auth header extraction; startup env detection.
93
94     ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
95     Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip
Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline
additionally:
96     1. **Captures** the source upload + the Gemini reel + the rally output
(intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-
consent auto-delete path.
97     2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw
video kept only for the training pool, not the no-consent path).
98     3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini
(`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks
the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
99     4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING
rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ?
the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
100     This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the
PERSONALIZATION contribution model): every consented free reel makes the owned model better.
Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-
retention gated by the D10 consent.
101
102     ## 6. Honest limits ? what this does NOT do / prove
103     - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity**
for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-
side, real-video).
104     - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a
CSV-byte claim.
105     - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with
**Gemini handoff ON** until the owned model clears a floor (open Q).
106     - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent**
gate ? both remain owner/counsel-gated before public, non-owner serving.
107
108     ## 7. Deliverables & progress tracker ? **source of truth**
109     Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off
`origin/master`, no stacking. Default-OFF where it touches core callers.
110
111     | ID | Deliverable | Depends | Gated? | Status | PR |
112     |----|-----|-----|-----|-----|----|
113     | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? |
**? APPROVED 2026-06-21** |
114     | M1 | `deployment.profile` + `compute_target` config + preset bundles +
precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? |
[#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
115     | M2 | `input_backend`/`input_root`; wire main CLI/API input through
`StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? |
[#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
116     | M3 | Configurable output/scratch base ? kill hardcoded `output/`
(`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden
+ stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |
[#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
117     | M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 |
safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/283) |
118     | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed
**runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | ? |

```

```

119 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) +
**containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
(GCP spend) | ? | ? |
120 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch
on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) |
M6 | ? owner (spend) | ? | ? |
121 | M8 | Per-job **cost ledger** (v_migration: `owner_id, gpu_active_seconds,
wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation +
`/api/.../cost` | M7 | ? owner (billing) | ? | ? |
122 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`;
DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |
123 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit
owner approval | ? | ? |
124 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the
upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-
eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the
golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the
owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent
gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
125 | **M12** | **gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the
source from + writes the stitched reel **back to the user's Google Drive next to the source**
(per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local
mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
126
127 **Testing strategy is a first-class deliverable** ?
[`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[`runlogs/`](runlogs/).
128
129 ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
130 ***? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see $2 D7?D12:**
131 1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
132 3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
133 4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to
proxy).
134 5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data;
counsel-gated to M9).
135 6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until
the owned model clears the floor; capture?store?eval?goldenize meanwhile).
136
137 **Still open (non-gating, smaller):**
138 2. **Stitch placement:** always co-located in the detect job (simplest, one metered
unit) vs split to a cheap CPU Cloud Run service when bursty? **Default co-located unless metrics
say otherwise.**
139 7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the
source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ?
distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
140 8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264
(determinism/parity)?
141 9. **Input cap:** is `<2GB` a hard client reject, or accept to the 4GB upload cap and
route larger files elsewhere?
142 10. **Directory name:** `COMPUTE_DECOUPLED_SERVING/` (chosen) vs keep
`CLOUD_RUN_GPU_SERVING/` ? confirm.
143
144 ## 9. Definition of done
145 - [ ] M1?M4 land (config + input + output-base + device layer), each green on golden
cross-OS + experiment no-regression.
146 - [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical
windows + segment ranges.
147 - [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog /
DEPLOY_REPORT** with a real sample run.
148 - [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth
(M9) gates multi-tenant.
149 - [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented
adult jobs; the owned model is on a measured path to the D11 floor.
150 - [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.

```

```

151
152     ## 10. References
153     **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014**
154     (`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`,
155     `HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`,
156     `LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:**
157     `docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-
158     ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code
159     anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`,
160     `stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`,
161     `backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`,
162     `backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-
163     mappers ? synthesis; 58 findings).
164
165     ---
166     ### Changelog
167     - 2026-06-21 ? created from the design workflow output; M0 in progress.
168     - 2026-06-21 ? **M0 APPROVED** ; **M1 landed**
169     ([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
170     `deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
171     (D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
172     change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach).
173     Suite 1068 green, cov 84.11%.
174     - 2026-06-21 ? **M2 landed** ([#280](https://github.com/Khelsutra/badminton-highlight-
175     indexer/pull/280)): `input_backend`/`input_root` + URI-aware main API/CLI input via
176     `resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
177     unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
178     on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
179     provenance. Suite 1105 green, cov 84.84%.
180     - 2026-06-21 ? **M3 landed** ([#282](https://github.com/Khelsutra/badminton-highlight-
181     indexer/pull/282)): configurable output/scratch base via
182     `backend/utils/paths.output_base(config)` ? the 3 segmenters' chunk/handoff/temp dirs root under
183     `output.output_dir` (default `"output"`, byte-identical; default-OFF ? no Launch Report). Fixes
184     the latent bug where scratch ignored `--output-dir`. Suite 1114 green, cov ~84.9%.
185     - 2026-06-21 ? **M4 landed** ([#283](https://github.com/Khelsutra/badminton-highlight-
186     indexer/pull/283)): `DeviceContext` + native WASB **CPU-fallback** via a new `device="auto"`
187     (cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade),
188     default unchanged. `"auto"` resolves before argv (never reaches `wasb_infer.py`); CUDA probe
189     cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126
190     green, cov 84.96%. **? M1?M4 (the config/input/output/device layer) COMPLETE.**
191
192     162

```

18. assistant (2026-06-21T09:25:27.665Z)

I now have everything I need. Let me confirm two specifics: the Gemini provider's `(segments, metrics)` return signature (for `gemini\_cost\_usd` capture in M8) and the config schema for any `deployment`/`security`/`storage` fields that M8/M9/M12 would extend. Let me also confirm there's truly no existing auth dependency or test for JWT.

19. user (2026-06-21T09:25:29.701Z)

```

backend\providers\base.py:10: def analyze_video(self, video_path: str, prompt: str,
backend\providers\gemini.py:38: def analyze_video(self, video_path: str, prompt: str,
backend\providers\gemini.py:58: metrics["oversize_skipped"] = True # callers
aggregate this into the run report
backend\providers\gemini.py:68: metrics["upload_time"] = time.time() -
t_upload_start
backend\providers\gemini.py:69: logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
backend\providers\gemini.py:105: metrics["process_time"] = time.time() - t_process_start
backend\providers\gemini.py:106: logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
backend\providers\gemini.py:127: metrics["gen_time"] = time.time() - t_gen_start
backend\providers\gemini.py:128: logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")

```

```

backend\providers\gemini.py:134:         metrics["generate_failed"] = True
backend\providers\gemini.py:146:         return segments, metrics
backend\pipeline\segmenters\gemini.py:218:         metrics["file_size_mb"] = chunk_size_mb
backend\pipeline\segmenters\gemini.py:219:         metrics["segments_found"] =
len(chunk_segments)
backend\pipeline\segmenters\gemini.py:242:         return chunk_label, chunk_segments,
metrics
backend\pipeline\segmenters\gemini.py:258:         all_metrics[label] = metrics
tests\test_gemini.py:61:     def analyze_video_side_effect(path, prompt, chunk_duration, label,
**kwargs):
tests\test_provider_gemini.py:8:     metrics['generate_failed'] so a FAILED chunk isn't silently
mistaken for 'no rallies'
tests\test_provider_gemini.py:52:         assert metrics["upload_time"] >= 0
tests\test_provider_gemini.py:53:         assert metrics["process_time"] >= 0
tests\test_provider_gemini.py:54:         assert metrics["gen_time"] >= 0
tests\test_provider_gemini.py:90:         assert metrics["oversize_skipped"] is True

```

20. user (2026-06-21T09:25:30.202Z)

```

backend\config\__init__.py
backend\config\presets.py
backend\config\loader.py
backend\config\models.py

```

21. user (2026-06-21T09:25:32.402Z)

```

---deps grep done---
pyproject.toml
requirements.txt

```

22. assistant (2026-06-21T09:25:38.051Z)

No JWT/crypto deps exist yet. The Gemini provider returns `(segments, metrics)` but `metrics` currently captures only timing (`upload\_time`, `process\_time`, `gen\_time`) ? no token/cost fields, so `gemini\_cost\_usd` capture would be net-new. Let me check the config models for the `deployment`/`storage`/`security` sections and the gemini provider's metrics dict to confirm what M8 can capture vs. what needs real prices.

23. user (2026-06-21T09:25:39.000Z)

```

1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):

```

```

25     min_resolution_width: int = 1280
26     min_resolution_height: int = 720
27     min_fps: float = 29.9
28     min_blur_threshold: float = 20.0
29     max_scene_cuts_per_minute: float = 0.5
30     relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
74         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
75         device: str = "cuda" # auto | cuda | mps | cpu
76         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
77         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
78         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
79         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
80         keep_frames: bool = False
81         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.

```

```

81     min_free_gb: float = 0.0
82     # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83     # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84     # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85     # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86     # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87     # Tri-state:
88     #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89     #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90     #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91     stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players
117        # < this. 0 = OFF.
118        min_players: int = Field(default=0, ge=0)
119        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120        court_polygon: Optional[list[list[float]]] = None
121        # Net line for the person two-sided-motion split (person features only ? the shuttle
122        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123        net_axis: Literal["x", "y"] = "y"
124        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)

```



```

132     # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133     # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135     detector_impl: str = "auto"
136     motion_threshold: float = 0.08
137     min_segment_duration: float = 3.0
138     dead_time_merge_gap: float = 2.0
139     chunk_duration: float = 90.0
140     chunk_overlap: float = 10.0
141     detector_model: str = "fasterrcnn_resnet50_fpn"
142     detector_score_threshold: float = 0.5
143     yolo_velocity_threshold: float = 0.15
144     yolo_frame_skip: int = 3
145     yolo_tracker: str = "bytetrack.yaml"
146     yolo_prefetch_workers: int = 2
147     min_action_window_duration: float = 2.0
148     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155     max_window_duration: float = 6.0
156     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
157     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160     # Only cuts (recall can't drop). OFF by default until measured/promoted.
161     window_hard_cap: bool = False
162     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164     # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169     window_inactivity_end: float = 1.5
170     inactivity_rally_thresh: float = 0.20
171     inactivity_min_density: float = 0.30
172     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.

```

```

175         # mahadevpura-1's ~65s residual ? 11x a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202         skip_ai_handoff: bool = False
203         # Optional debug knob: trim every video to the first N seconds before processing.
204         debug_duration: Optional[float] = None
205
206         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208         fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210         @field_validator("default_segmenter")
211         @classmethod
212         def segmenter_must_be_registered(cls, v: str) -> str:
213             # Registry check happens at runtime in main/segmenter selection.
214             # Here we just allow any string for forward compatibility.
215             return v
216
217         @model_validator(mode="after")
218         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220             # step makes `while t < duration: t += step` loop forever, growing memory before
any
221             # GPU work. Reject the bad config at load time instead.
222             if self.chunk_overlap >= self.chunk_duration:
223                 raise ValueError(
224                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"

```

```

225         f"({self.chunk_duration}); otherwise chunking never advances."
226     )
227     return self
228
229
230 class OutputConfig(BaseModel):
231     default_highlights_name: str = "highlights.mp4"
232     db_name: str = "sports_indexer.db"
233     # Directory where the DB, highlight reels, and processed clips are written.
234     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235     output_dir: str = "output"
236
237
238 class WatermarkConfig(BaseModel):
239     """Branding overlay burned into each highlight clip during the per-clip re-encode
240     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241     (under `stitching.watermark` in config.json) to turn it off. The text is fully
242     configurable. The concat stage stays stream-copy (-c copy)."""
243     enabled: bool = True
244     text: str = "Generated by Khelsutra.guru"
245     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
247     font: str = "Sans" # fontconfig family used when font_path is empty
248     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
249     # fraction of frame height
250     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
251     box: bool = True # faint backing box behind the text for legibility
252     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
253     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
254     right"
255
256 class StitchingConfig(BaseModel):
257     begin_padding: float = 0.5
258     end_padding: float = 0.5
259     use_cache: bool = False
260     min_shot_count: int = 1
261     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
262     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
263     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
264     # the
265     # fully-local no-AI path, whereas duration is derived from the rally's own
266     # start/end. The
267     # local detector over-fires and its false positives are mostly SHORT (motion blips,
268     # background-court fragments), so this keeps the reel to the eye-catching long
269     # rallies.
270     # OFF by default ? the detector output is unchanged, only which rallies reach the
271     # reel.
272     min_rally_duration: float = Field(default=0.0, ge=0.0)
273     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
274
275 class LoggingConfig(BaseModel):
276     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
277     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
278     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
279     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
280     # them to WARNING; set true to restore full chatter for request-flow debugging.
281     verbose_http: bool = False
282
283 class SecurityConfig(BaseModel):
284     # When set, /api/process video_path must resolve under this directory (hosted/tenant

```

```

282     # mode). Default None preserves the single-user local 'any local file' workflow.
283     allowed_video_root: Optional[str] = None
284
285     # --- Chunked upload (B2, PR-2) ---
286     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288     # contain upload_dir or /api/process will reject the uploaded paths.
289     upload_dir: str = "output/uploads"
290     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291     # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292     max_upload_mb: int = 4096
293     max_part_mb: int = 95
294     allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296
297     class StorageConfig(BaseModel):
298         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
299         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
300         Spaces
301         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
302         locally MOUNTED
303         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
304         the constructor ?
305         **secrets are never stored here**, only endpoints / regions / file paths / source-
306         selectors
307         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
308         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
309         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
310         breaking).
311         output_root: str = ""
312         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
313         `backend`/`output_root`).
314         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
315         read IN PLACE
316         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
317         bare/relative
318         # input name is resolved against ``input_root`` (e.g.
319         ``input_root="gs://bucket/videos"`` + "x.mp4"
320         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
321         ``input_backend``
322         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
323         input
324         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
325         are fetched to a
326         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
327         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
328         input_root: str = ""
329         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
330         # One folder per video keyed by the MD5 video_id:
331         <root>/[<workspace_prefix>]/<video_id>/...
332         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
333         root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
335         When OFF
336         # the legacy flat layout is used unchanged.
337         single_folder_per_video: bool = False
338         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
339         workspace_root: str = ""
340         # Cloud namespace prefix under the root so per-video folders sit tidily beside
341         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
342         directly.
343         workspace_prefix: str = "workspaces"
344         local: dict[str, Any] = Field(default_factory=dict)
345         s3: dict[str, Any] = Field(default_factory=dict)
346         gcs: dict[str, Any] = Field(default_factory=dict)

```

```

331         gdrive: dict[str, Any] = Field(default_factory=dict)
332
333
334     class DeploymentConfig(BaseModel):
335         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
336         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
337
338         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
339         behaves exactly as before this section existed. A profile is opt-in (``config.json``
340         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
341         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
342         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
343         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
344
345         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
346
347         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
348         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
349         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
350         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
351         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
352
353
354     class AppConfig(BaseModel):
355         """Root configuration object.
356
357         All runtime configuration should be accessed via an instance of this class
358         (or the loader) rather than raw dict lookups.
359         """
360
361         sport: Sport = "badminton"
362         ai_provider: str = "gemini"
363         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
364         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
365         ai_model: str = "gemini-flash-latest"
366
367         validation: ValidationConfig = Field(default_factory=ValidationConfig)
368         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
369         output: OutputConfig = Field(default_factory=OutputConfig)
370         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
371         security: SecurityConfig = Field(default_factory=SecurityConfig)
372         logging: LoggingConfig = Field(default_factory=LoggingConfig)
373         storage: StorageConfig = Field(default_factory=StorageConfig)
374         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
375
376         # Allow extra keys during transition so existing config.json files do not break.
377         model_config = {
378             "extra": "allow",
379             "validate_assignment": True,
380         }
381
382         # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
383         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
384         # attribute access / the typed config. A nested *section* is returned as a plain
dict

```

```

385     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
386     # section model instead breaks it (a model has no .get), which bit every validator
on
387     # real runs. Scalar leaves are returned as-is.
388     def get(self, key: str, default: Any = None) -> Any:
389         _MISSING = object()
390         val: Any = _MISSING
391         if key in type(self).model_fields or hasattr(self, key):
392             val = getattr(self, key, _MISSING)
393         if val is _MISSING:
394             # Legacy convenience: find a leaf key inside any nested section. Sections
are
395             # discovered dynamically from the model fields, so a future top-level
section
396             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
397             for fname in type(self).model_fields:
398                 section = getattr(self, fname, None)
399                 if isinstance(section, BaseModel) and hasattr(section, key):
400                     val = getattr(section, key)
401                     break
402             if val is _MISSING:
403                 return default
404             if isinstance(val, BaseModel):
405                 return val.model_dump(mode="json")
406             return val
407
408     def __getitem__(self, key: str) -> Any:
409         val = self.get(key)
410         if val is None and key not in type(self).model_fields:
411             # Re-raise like a real dict for unknown top-level
412             raise KeyError(key)
413         return val
414
415
416     # Convenience type alias for callers that want to annotate
417     Config = AppConfig
418

```

24. user (2026-06-21T09:25:39.940Z)

```

38     def analyze_video(self, video_path: str, prompt: str,
39                       chunk_duration: float = 0.0,
40                       label: str = "") -> Tuple[list, dict]:
41         log_prefix = f"Gemini {label}" if label else "Gemini"
42
43         metrics = {
44             "upload_time": 0.0,
45             "process_time": 0.0,
46             "gen_time": 0.0
47         }
48
49         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
50         try:
51             size_mb = os.path.getsize(video_path) / (1024 * 1024)
52         except OSError:
53             size_mb = 0.0
54         if size_mb > MAX_UPLOAD_MB:
55             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? ")
56             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
57             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
58         metrics["oversize_skipped"] = True # callers aggregate this into the run

```

```

report
59         return [], metrics
60
61     # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
62     logger.info(f"[{log_prefix}] Uploading {video_path}...")
63     t_upload_start = time.time()
64     video_file = None
65     for attempt in range(3):
66         try:
67             video_file = self.client.files.upload(file=video_path)
68             metrics["upload_time"] = time.time() - t_upload_start
69             logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
70             break
71         except Exception as e:
72             logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
73             if attempt < 2:
74                 time.sleep(2 * (attempt + 1))
75     if video_file is None:
76         logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
77         return [], metrics
78
79     # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
80     # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
81     t_process_start = time.time()
82     process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
83
84     def _processed():
85         nonlocal video_file
86         video_file = self.client.files.get(name=video_file.name)
87         return None if video_file.state.name == "PROCESSING" else video_file
88
89     try:
90         video_file = poll_until(_processed, process_policy,
91                                label=f"[{log_prefix}] processing", logger=logger)
92     except PolicyTimeout:
93         logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
94         self._safe_delete(video_file)
95         return [], metrics
96     except Exception as e:
97         logger.error(f"[{log_prefix}] Status check failed: {e}")
98         self._safe_delete(video_file)
99         return [], metrics
100     if video_file.state.name == "FAILED":
101         logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
102         self._safe_delete(video_file)
103         return [], metrics
104
105     metrics["process_time"] = time.time() - t_process_start
106     logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
107
108     # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
109     # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
110     # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).

```

```

111         logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
112         t_gen_start = time.time()
113         gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
114         raw_text = None
115
116         def _generate():
117             return self.client.models.generate_content(
118                 model=self.model_name,
119                 contents=[video_file, prompt],
120                 config=types.GenerateContentConfig(
121                     response_mime_type="application/json", temperature=0.0),
122             )
123
124         try:
125             response = run_bounded(_generate, gen_policy, label=f"[{log_prefix}
generate", logger=logger)
126             raw_text = response.text
127             metrics["gen_time"] = time.time() - t_gen_start
128             logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
129             except Exception as e:                # PolicyTimeout (hang) or the last 503 ?
both end here
130             logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
131             # Flag the failure so the caller can tell "chunk FAILED (no analysis)" apart
from "chunk
132             # genuinely had 0 rallies" ? otherwise a timed-out/503'd chunk silently
drops its time
133             # range from the result (a partial reel that looks complete). Mirrors
oversize_skipped.
134             metrics["generate_failed"] = True
135             self._safe_delete(video_file)
136
137             if not raw_text:
138                 return [], metrics
139
140             # 4. Parse JSON
141             try:
142                 segments = json.loads(raw_text)
143                 if not isinstance(segments, list):
144                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
145                     return [], metrics
146                 return segments, metrics
147             except Exception as e:

```

25. assistant (2026-06-21T09:25:52.283Z)

I now have complete grounding. Let me capture the precise facts and emit the structured output.

Key confirmed facts:

- **DB schema is at v5** (`database.py:103-104`, `_migrate_to_v5` sets `PRAGMA user_version = 5`); next migration is v6. The ladder uses `_add_column_idempotent` (`database.py:61-77`) and a versioned `if version < N` block (`database.py:91-106`).
- **v4 added NULLABLE `user_id` to `videos` + `candidate_segments`** (`database.py:227-237`) ? explicitly the "per-tenant scoping" groundwork, NULL=local. But the `jobs` table (v2, `database.py:188-203`) has NO `owner_id`/cost columns** and NO `user_id` ? confirmed by the design doc (`05-COST-ATTRIBUTION.md:7`).
- **No auth code exists anywhere** ? CORS is wide open with `allow_credentials=False` (`main.py:42-48`); the comment at `main.py:40-41` explicitly says this default is "unsafe to carry into the auth/tenancy work." No JWT/JWKS/Cf-Access/Authorization handling. No PyJWT/cryptography/jose dep in requirements.
- **Gemini provider returns `(segments, metrics)`** (`gemini.py:38-40,146`) but `metrics` is timing-only (`upload_time`/`process_time`/`gen_time`, `gemini.py:43-47`) ? NO token/cost. So `gemini_cost_usd` needs either new token capture or a placeholder.

- **StorageBackend ABC** (`base.py:17-49`) has `signed_url` with `NotImplementedError` default; GCS implements it (`gcs.py:61-63`). `gdrive.py` is mount-only and explicitly does NOT touch the Drive API (`gdrive.py:1-21`). Credentials convention: "secrets never in config" (`__init__.py:98`, `StorageConfig` docstring `models.py:297-303`, GCS uses ADC `gcs.py:4-6`).
- Config has `SecurityConfig` (`models.py:280-294`), `StorageConfig` with per-backend loose dicts incl. `gdrive: dict` (`models.py:328-331`), `DeploymentConfig` (`models.py:334-351`).

26. user (2026-06-21T09:27:12.910Z)

Structured output provided successfully

27. assistant (2026-06-21T09:27:31.610Z)

I've mapped all four gates grounded in the actual code and design doc. Summary of the key code-reality findings for your area:

M8 (cost ledger) ? `creds_calibration`, HIGH unblock value. Nearly everything is codeable now: the v6 migration (the `jobs` table at `database.py:188-203` has zero cost columns; mirror the v4 `NULLable-user_id` additive pattern at `database.py:219-237` using `_add_column_idempotent`), the `pricebook`/`training_contributions` tables, the `cost_usd = ? usage * rate` calculator with USD?INR display, worker-side `gpu_active_seconds`/`wall_seconds` timing in `job_worker.py:72-97`, and the `/api/.../cost` endpoints. All default-OFF behind a seeded `placeholder-v0` pricebook with `$0` rates. The owner prerequisite is purely **real calibrated prices** (GCP \$/GPU-sec, egress, FX rate, margin) plus the flip-to-live OK. Caveat I surfaced: `gemini_cost_usd` has no real source ? `gemini.py` metrics is timing-only (`gemini.py:43-47,146`), so token-cost stays a placeholder until provided.

M9 splits cleanly into two gates:

- **M9-auth ? `creds_calibration`, HIGH.** The Cf-Access JWT verifier + `user_id` scoping is fully writable and unit-testable now with an in-test RSA keypair + fake JWKS (zero CF account, zero network). No JWT/crypto dep exists today (must add `PyJWT[crypto]`); no auth code exists anywhere ? CORS is wide-open and the code itself flags it as "unsafe to carry into the auth/tenancy work" (`main.py:40-48`). The v4/v5 `user_id` columns already exist (`database.py:227-237,256-285`); only `jobs` still needs the column (folds into the M8 v6 migration). Owner prerequisite is just CF config **strings** (team-domain/JWKS issuer, AUD) + the ingress-lock deploy step.
- **M9-consent ? `counsel_consent`, LOW.** Not codeable to completion. Only the default-OFF no-train mechanism (consent flag column + capture-vs-auto-delete branch defaulting to delete) is safe to build. The ToS/DPA text, adults-only attestation rule, and the flip-to-live-training are legal judgments (D10, `README:37`; CLAUDE.md minors/training guardrails). This should NOT block M9-auth, which is independent.

M12 (gdrive-api OAuth) ? `creds_calibration`, MEDIUM. The `GDriveApiStorage(StorageBackend)` is fully buildable now against a **mocked/injected Drive service** (mirroring the GCS client-injection test seam at `gcs.py:18-25`), registered in `_backend_class` (`storage/__init__.py:77-91`) under a new `gdrive-api://` scheme that leaves the mount-only `gdrive.py` (`gdrive.py:1-21`, explicitly no Drive API) untouched for truly-local. Owner prerequisite is a **Google OAuth client/creds + consent screen + per-user tokens** ? none needed to write or unit-test the class. Live per-user write-back overlaps M9-consent, so it shouldn't ship ahead of the consent gate.

The single highest-leverage observation: **M8-ledger and M9-auth together unblock a large independent PR surface with mocked tests and zero owner involvement** ? the owner's only blocking actions reduce to (a) a one-row pricebook insert with real rates + FX/margin, and (b) a handful of Cloudflare config strings + an ingress-lock. M9-consent (counsel) and M12-live-OAuth (creds) are the genuinely owner-bound pieces.